

Centro Universitário de João Pessoa – Unipe
Ciência da Computação
Computabilidade e Complexidade de Algoritmos

Docente: Carlos Herriot

Discentes:

Beatriz Almeida de Souza Silva

Gabriel Bianchini Carvalho

José Carlos de Oliveira Neto

Lucas Amorim Pontes Sousa Lima

Rigel Silva de Souza Sales

Ativ. unidade 1: Análises de Complexidade de Funções

Implementações das funções em pseudocódigo e cálculos de suas complexidades.

O pseudocódigo foi colocado como um arquivo markdown no repositório, e prints das funções estão disponíveis aqui. Os cálculos foram feitos em papel, com as complexidades das linhas sendo mostradas em blocos, com indicação do que cada bloco é.

Função 1:

```
1 count_distinct_occurrences:
2     inteiro n, k, occurrences = 0
3
4     escreva ""
5     original_array = array_starter(n)
6
7     escreva ""
8     search_array = array_starter(k)
9
10    para i variando de 1 ate n faça:
11        para j variando de 1 até k faça:
12            se original_array[i] = search_array[j]:
13                occurrences = occurrences + 1
14            fimse
15        fimpara
16    fimpara
17
18    escreva ""
19    escreva ""
20    escreva ""
21    escreva ""
22    escreva ""
23    escreva ""
24
25    free(original_array)
26    free(search_array)
```

1 } atribuições

1 } n

1 } n

n+1 } "para" aninhado

n(k+1) } "se"

n*k

1 } escreva do resultado (6)

1 } 1

1 } 1

1 } 1

1 } 1

1 } 1

n } frees (2n)

1 + 2n + 2 + n + 1 + n(k+1) + nk + 6 + 2n

3n + 4 + nk + n + nk + 6 + 2n

2nk + 6n + 6

O(nk)

Função 2:

1. } atribucões

$\left. \begin{matrix} n^2 \\ n^2 \end{matrix} \right\}$ criação e transando a matriz $(2n^2)$

$n+1$
 $n(n+1)$
 n^2
 n^2

"para" aninhado,
 "se", e linha de
 fim do "se".

$\left. \begin{matrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{matrix} \right\}$ valores do resultado (6)

$\left. \begin{matrix} n+1 \\ n \\ 1 \end{matrix} \right\}$ "para" e
"frees"

$$1 + 2n^2 + n + 1 + \underbrace{n(n+1)} + \underbrace{n^2 + n^2} + 6 + n + 1 + n + 1$$

$$2n^2 + n + 2 + n^2 + n + 2n^2 + 2n + 8$$

$$5n^2 + 4n + 10$$

$O(n^2)$

Função 3:

(código na próxima página)

$\left. \begin{matrix} 1 \\ 1 \\ 1 \end{matrix} \right\} \text{atribuições (3)}$
 $\left. \begin{matrix} 1 \\ n^3 \\ n^3 \\ n^3 \end{matrix} \right\} \begin{matrix} (3 + 2n^3) \\ \text{criando e} \\ \text{mostrando} \\ \text{matriz A} \end{matrix}$
 $\left. \begin{matrix} n+1 \\ n(n+1) \\ n^2(n+1) \\ n^2(n+1) \end{matrix} \right\} \begin{matrix} \text{"para" ani-} \\ \text{nhado para} \\ \text{somar ele-} \\ \text{mentos de} \\ \text{A} \end{matrix}$
 $\left. \begin{matrix} 1 \\ 1 \end{matrix} \right\} \text{escovas (2)}$
 $\left. \begin{matrix} n^3 \\ n^3 \\ n^3 \end{matrix} \right\} \begin{matrix} (2 + 2n^3) \\ \text{criando e} \\ \text{mostrando} \\ \text{matriz B} \end{matrix}$
 $\left. \begin{matrix} n+1 \\ n(n+1) \\ n^2(n+1) \\ n^2(n+1) \end{matrix} \right\} \begin{matrix} \text{"para" ani-} \\ \text{nhado para} \\ \text{somar ele-} \\ \text{mentos de} \\ \text{B} \end{matrix}$

$\left. \begin{matrix} 1 \\ 1 \end{matrix} \right\} \text{escovas (2)}$
 $\left. \begin{matrix} 1 \\ 1 \\ 1 \\ 1 \end{matrix} \right\} \begin{matrix} \text{"se" de checa-} \\ \text{sem da resposta} \end{matrix} \quad (4)$
 $\left. \begin{matrix} 1 \\ 1 \\ 1 \\ 1 \end{matrix} \right\} \text{escovas da resposta (5)}$
 $\left. \begin{matrix} n^3 \\ n^3 \end{matrix} \right\} \text{freas. } (2n^3)$

(Função 3:
 Comparação de
 métodos tri-
 dimensionais)

$$\begin{aligned}
 & 3 + 3 + 2n^3 + n + 1 + \cancel{n(n+1)} + \cancel{n^2(n+1)} \\
 & + \cancel{n^2(n+1)} + 2 + 2 + 2n^3 + n + 1 + \cancel{n(n+1)} \\
 & + \cancel{n^2(n+1)} + \cancel{n^2(n+1)} + 2 + 4 + 5 + 2n^3 \\
 & n^3 + n + n^3 + n + n^3 + n + n^3 + n \\
 & (4n^3 + 4) \\
 & \cancel{3 + 3 + 2n^3 + n + 1 + n^2 + n} + \cancel{4n^3 + 4} \\
 & + \cancel{4 + 2n^3 + n + 1 + n^2 + n} + \cancel{11 + 2n^3} \\
 & 10n^3 + 2n^2 + 4n + 27
 \end{aligned}$$

$O(n^3)$

(Função 3)

```
1  matrizes_comparison:
2      inteiro n, answer
3      inteiro sum_a = 0
4      inteiro sum_b = 0
5
6      escreva ""
7      A = matrix_3d_starter(n)
8      escreva ""
9      show_3d_matrix(A, n)
10     escreva ""
11
12     para i variando de 1 até n faça:
13         para j variando de 1 até n faça:
14             para k variando de 1 até n faça:
15                 sum_a = sum_a + A[i][j][k]
16             fimpara
17         fimpara
18     fimpara
19
20     escreva ""
21     escreva ""
22
23     B = matrix_3d_starter(n)
24     escreva ""
25     show_3d_matrix(B, n)
26     escreva ""
27
28     para i variando de 1 até n faça:
29         para j variando de 1 até n faça:
30             para k variando de 1 até n faça:
31                 sum_b = sum_b + B[i][j][k]
32             fimpara
33         fimpara
34     fimpara
35
36     escreva ""
37     escreva ""
38
39     se sum_a >= sum_b:
40         answer = 1
41     senão:
42         answer = 0
43     fimse
44
45     escreva ""
46     escreva ""
47     escreva ""
48     escreva ""
49     escreva ""
50
51     free_3d_matrix(A, n)
52     free_3d_matrix(B, n)
```

Função 4:

```

1 process_array:
2     inteiro n, sum = 0
3
4     V = array_starter(n)
5     show_array(V, n)
6
7     para i variando de 1 até n faça:
8         se V[i] % 2 = 0:
9             sum = sum + V[i]
10        senão:
11            inteiro factorial = 1
12
13            para j variando de 1 até V[i] faça:
14                factorial = factorial * j
15            fimpara
16
17            sum = sum + factorial
18        fimse
19    fimpara
20
21    escreva ""
22    escreva ""
23    escreva ""
24    escreva ""
25    escreva ""
26    escreva ""
27
28    free(V)

```

1 } atribuição
n } escreva (2n)
n }
n+1 → 1º para
n } se e senão (2n+1)
1 }
n }
n(n+1) → 2º para
n² → fatorial
n → atribuição
1 }
1 }
1 } escreva da (6)
1 } resposta
1 }
n → free

7 + 2n + n + 1 + 2n + 1 + n(n+1) + n² + n + n
9 + 5n + n² + n + n² + 2n
2n² + 8n + 9 $O(n^2)$

Função 4:
Análise de
casos. assi-
métricos no
condicional

© teacherprintable.com

CS CamScanner

Função 5:

```

1 count_elements_in_sorted_array:
2     inteiro n, count = 0
3
4     escreva ""
5     A = array_starter(n)
6     show_array(A, n)
7
8     escreva ""
9     B = array_starter(n)
10    show_array(B, n)
11
12    escreva ""
13    escreva ""
14    bubble_sort(B, n)
15    show_array(B, n)
16
17    para i variando de 1 até n faça:
18        se binary_search(B, n, A[i]):
19            count = count + 1
20    fimse
21
22    escreva ""
23    escreva ""
24    escreva ""
25    escreva ""
26    escreva ""
27    escreva ""
28
29    free(A)
30    free(B)

```

Handwritten analysis of the code's complexity:

- Array creation and modification:**
 - 1: $(2n + 1)$ criando e modificando valor A
 - n: $(2n + 1)$ criando e modificando valor B
- Sorting:**
 - 1: $(n^2 + n + 2)$ escrevendo
 - n: ordenando e modificando valor
- Binary Search:**
 - n+1: "para" para percorrer vetor
 - n: "se" para busca binária
- Output:**
 - 1: (6) impressões da função
 - n: $(2n)$ free's

Complexity Calculations:

- $1 + 4n + 2 + n^2 + n + 2 + n + 1 + n \cdot \log n + n + 6 + 2n$
- $n \log n + n^2 + 8n + 12$
- Complexity:** $O(n^2)$
- Função 5:** Contagem de elementos presentes em vetor ordenado.

Tempos de execução, com cálculos abaixo:

Nome da função	Big O	Tempo de execução
1. count_distinct_occurrences	$O(nk)$	2 seg
2. matrix_pair_analysis	$O(n^2)$	0.0025 seg
3. matrices_comparison	$O(n^3)$	0.27 seg
4. process_array	$O(n^2)$	25 seg
5. count_elements_in_sorted_array	$O(n^2)$	1.000.000 seg

$10^8 = 100\,000\,000$ (Cálculos de tempo)

Função 1:
 $n = 50\,000$, $K = 4\,000$
 $O(nK) \rightarrow 50\,000 \cdot 4\,000 = \frac{200\,000\,000}{100\,000\,000}$
 $= t = 2 \text{ segundos}$

Função 2:
 $n = 500$
 $O(n^2) \rightarrow 500^2 = \frac{250\,000}{100\,000\,000} = 0,0025 \text{ segundos}$

Função 3:
 $n = 300$
 $O(n^3) = 300^3 = \frac{27\,000\,000}{100\,000\,000} = 0,27 \text{ segundos}$

Função 4:
 $n = 50\,000$
 $O(n^2) = 50\,000^2 = \frac{2\,500\,000\,000}{100\,000\,000} = 25 \text{ segundos}$

Função 5:
 $n = 10\,000\,000$
 $O(n^2) = 10\,000\,000^2 = \frac{100\,000\,000\,000\,000}{100\,000\,000} = 1\,000\,000 \text{ segundos}$

© peachprintable.com

CS CamScanner

Foi necessário fazer os cálculos também para as principais funções auxiliares implementadas, para saber seu tempo de execução quando elas são chamadas nas funções principais. Os resultados estão na seguinte tabela:

Obs.: r = rows, c = columns (linhas x colunas), l = layers (camadas).

Nome da função	Big O	Observação
auto_matrix_generator	$O(n^2)$	$O(r \times c)$, caso n° linhas $\neq$ n° colunas
input_matrix_generator	$O(n^2)$	$O(r \times c)$, caso n° linhas $\neq$ n° colunas
show_matrix	$O(n^2)$	$O(r \times c)$, caso n° linhas $\neq$ n° colunas
matrix_starter	$O(n^2)$	$O(r \times c)$, caso n° linhas $\neq$ n° colunas

auto_3d_matrix_generator	$O(n^3)$	$O(l \times r \times c)$, caso n° camadas $\neq n^\circ$ linhas $\neq n^\circ$ colunas
input_3d_matrix_generator	$O(n^3)$	$O(l \times r \times c)$, caso n° camadas $\neq n^\circ$ linhas $\neq n^\circ$ colunas
matrix_3d_starter	$O(n^3)$	$O(l \times r \times c)$, caso n° camadas $\neq n^\circ$ linhas $\neq n^\circ$ colunas
show_3d_matrix	$O(n^3)$	$O(l \times r \times c)$, caso n° camadas $\neq n^\circ$ linhas $\neq n^\circ$ colunas
free_3d_matrix	$O(n^3)$	$O(l \times r \times c)$, caso n° camadas $\neq n^\circ$ linhas $\neq n^\circ$ colunas
auto_array_generator	$O(n)$	-
input_array_generator	$O(n)$	-
show_array	$O(n)$	-
array_starter	$O(n)$	-
bubble_sort	$O(n^2)$	-
binary_search	$O(\log n)$	-